JSPM UNIVERSITY PUNE

Inspiring Innovation. Transforming Talent.

Faculty of Science and Technology

School of Computational Sciences

Class: S. Y. B.Tech. (CSE)

Course Name: _____

Course Code: _____

Semester: IV Academic Year: 2024-25

Name:	
Division:	
PRN No.:	

INDEX

PRN No. : _____ Course Name: Object Oriented Programming Lab

Course Code : 230GCSB14_04

Exp. No.	Title of Experiment	Page No.	Date	Sign of Course Teacher
01.	Write a C++ program to find the greatest of three numbers.			
02.	Write a C++ program to display Fibonacci series.			
03.	Write a C++ program to create student class with member functions to accept student data as an user input and print the student details. Create an object to access student data.			
04.	Write a C++ program to swap two numbers with the help of object-oriented concepts.			
05.	Write a C++ program to find area of a circle, triangle and rectangle using function overloading.			
06.	Write a C++ programs to design a BankAccount class with methods like deposit, withdraw, and getBalance. Ensure you have a constructor to set an initial balance.			
07.	Write a C++ program to create a base class Account with attributes accountNumber and balance. Derive classes SavingsAccount and CurrentAccount with attributes interestRate and overdraftLimit. Add methods to deposit, withdraw, and display details.			
08.	Write a C++ program to demonstrate pointers and virtual functions. Create a base class Shape with a virtual area() function and derive Circle and Rectangle classes. Use a base class pointer to call the area() method and display the result.			
09.	Implement a C++ program with a function divide() that handles division by zero using exception handling. If a division by zero occurs, throw an exception and display an error message.			
10.	Implement a C++ program to create a template function findMax() that returns the maximum of two values of any data type (e.g., int, float, string)			
11.	Implement a C++ program to read and display student details (name, age, grade) from a file. If the file doesn't exist, display an error. Also, write new student details to a file.			

Lab Exercise-01

Title:

Write a C++ program to find the greatest of three numbers.

Aim:

To write a C++ program that determines and displays the greatest of three given numbers by comparing their values.

Learning objectives:

- Understand the use of conditional statements in C++.
- Learn how to compare multiple values logically.
- Develop the ability to implement decision-making structures in a program.
- Familiarize with basic input/output operations in C++.

Learning outcomes:

- Write and execute a C++ program that uses conditional statements.
- Identify and apply the appropriate logic to solve comparison-based problems.
- Debug and test a program effectively for accuracy.
- Demonstrate an understanding of structured programming concepts.

OOP LAB EXERCISE_01

Requirements:

Hardware Requirements:

- A computer system with a minimum of 2GB RAM.
- Processor: Intel i3 or equivalent and above.
- Monitor, keyboard, and mouse.

Software Requirements:

- Operating System: Windows/Linux/macOS
- IDE: Code::Blocks, Dev-C++, or any other C++ compiler.

Prerequisites:

- Basic understanding of C++ syntax.
- Familiarity with conditional statements (if, if-else).
- Knowledge of standard input/output functions.

Theory:

- Conditional Statements in C++
- Conditional statements are used to perform different actions based on different conditions. The main conditional constructs in C++ are:
- if statement: Executes a block of code if a specified condition is true.
- if-else statement: Executes one block of code if a condition is true and another if it is false.
- else if ladder: Used to check multiple conditions sequentially.
- Logic to Find the Greatest of Three Numbers:
- Compare the three numbers using conditional statements.

- Check if the first number is greater than or equal to the second and third numbers.
- If not, check whether the second number is greater than or equal to the first and third numbers.
- Otherwise, the third number is the greatest.

Methodology:

- Start by including the necessary header file (#include<iostream>).
- Declare three variables to store the numbers.
- Take input from the user for all three numbers.
- Use conditional statements to compare the numbers and determine the greatest one.
- Display the result.
- Test the program with different inputs to verify its correctness.

Algorithm:

1. Start.
2. Include the header file: #include<iostream>.
3. Declare three variables: num1, num2, num3.
4. Prompt the user to input three numbers.
5. Read the three numbers using cin.
6. Compare the numbers using conditional statements:
   a. If num1 > num2 and num1 > num3, print num1 as the greatest.
   b. Else if num2 > num1 and num2 > num3, print num2 as the greatest.
   c. Else, print num3 as the greatest.
7. End.

Program:

```
#include <iostream>
using namespace std;
 int main()
{
int num1, num2, num3;
// Prompt user for input
cout << "Enter the first number: "; cin >> num1;
cout << "Enter the second number: "; cin >> num2;
cout << "Enter the third number: "; cin >> num3;
// Determine the greatest number
if (num1 >= num2 && num1 >= num3)
{
cout << "The greatest number is: " << num1 << endl;
}
else if (num2 >= num1 && num2 >= num3)
{
cout << "The greatest number is: " << num2 << endl;
}
else
{
cout << "The greatest number is: " << num3 << endl;
}
return 0;
```

}

Output:

Conclusion:

This exercise demonstrates how to use conditional statements and relational operators in C++ to solve decision-making problems. By writing a program to find the greatest of three numbers, I have gained hands-on experience in implementing logical conditions and handling user inputs/outputs.

References:

- "The C++ Programming Language" by Bjarne Stroustrup
- "Object-Oriented Programming with C++" by E. Balagurusamy
- "C++ Primer" by Stanley B. Lippman, Josée Lajoie, and Barbara E. Moo

Lab Exercise-02

Title:

Write a C++ program to display Fibonacci series.

Aim:

The aim of this lab exercise is to write a C++ program that generates and displays the Fibonacci series up to a specified number of terms.

Theory:

Fibonacci Series

The Fibonacci series is a sequence of numbers where each number is the sum of the two preceding ones, usually starting with 0 and 1. The sequence commonly starts as follows:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

The mathematical formula for the Fibonacci series can be represented as:

- $F(0) = 0$
- $F(1) = 1$
- $F(n) = F(n-1) + F(n-2)$ for $n > 1$

C++ Programming Constructs

- Loops: Used to execute a block of code repeatedly. In this exercise, a while or for loop can be utilized.
- Conditionals: Statements that perform different actions based on whether a condition is true or false.
- Variables: Used to store data values. In this case, variables will be needed to store the current and previous Fibonacci numbers.

Methodology:

The program will generate the Fibonacci series by:

- Asking the user to input the number of terms they wish to display.
- Using an iterative approach (using a loop) to calculate each Fibonacci number.
- Displaying the Fibonacci sequence on the console.

The logic behind the Fibonacci sequence is implemented by storing the first two terms (0 and 1) and then using a loop to calculate each subsequent term by adding the previous two.

Algorithm:

1. Start.
2. Declare variables to store the first two Fibonacci numbers, a and b, initialized to 0 and 1, respectively.
3. Prompt the user to enter a maximum number for the series.
4. Print the initial numbers (0 and 1).
5. Use a loop to generate Fibonacci numbers

While the next number is less than or equal to the specified maximum:

- Calculate the next Fibonacci number as the sum of a and b.
- Print the next number.
- Update a and b (set a to b and b to the next number).

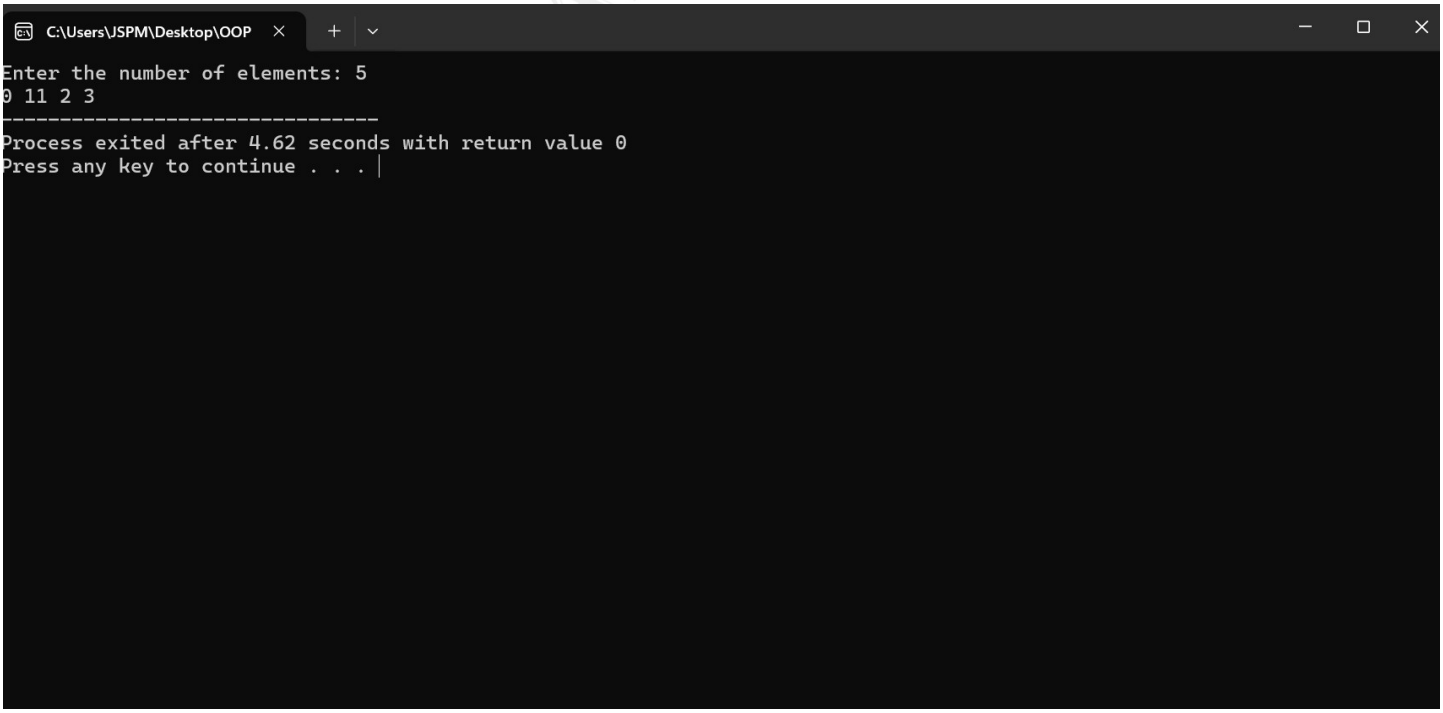
6. End the program.

Program:

```
#include <iostream> using
namespace std; int main()
{
 int n1=0,n2=1,n3,i,num;
 cout<<"Enter the number of elements: "; cin>>num;
 cout<<n1<<" "; cout<<n2;
 for(i=2;i<num;i++)
 {
```

```
  n3=n1+n2; cout<<n3<<" ";
  n1=n2;
  n2=n3;
 }
   return 0;
}
```

Output:



Conclusion

In this lab exercise, I have successfully written a C++ program to display the Fibonacci series. I have learned how to implement loops to generate and display

sequences, write clean and effective C++ code for generating a mathematical sequence.

References:

- "The C++ Programming Language" by Bjarne Stroustrup
- "Object-Oriented Programming with C++" by E. Balagurusamy
- "C++ Primer" by Stanley B. Lippman, Josée Lajoie, and Barbara E. Moo

Lab Exercise-03

Title:

Write a C++ program to create student class with member functions to accept student data as an user input and print the student details. Create an object to access student data.

Aim:

To write a C++ program that create student class with data member and to access student data as an user input and print that details and create object for access that student data.

Theory

Classes and Objects in OOP

- Class is a blueprint or template for creating objects. A class defines a set of attributes (data members) and behaviours (member functions or methods) that its objects will have.
- Object is an instance of a class. It is created based on the class definition, and each object has its own set of values for the attributes defined in the class.

Program construct:

- Define the Student Class
- Input Functionality (inputStudentDetails)
- Output Functionality (printStudentDetails)
- The main() function is where the program starts executing.

Methodology:

- Start by including necessary header file.
- Create a student class accept with data members and member function.
- Take an input from user for student details.
- Create function for accept and display data.
- Create object to fetch data.

Algorithm:

1. Start
2. Define the Student class
3. Create the main function
4. Call the inputStudentDetails() function
5. Call the printStudentDetails() function:

   Display the following details of the student:

   - Name
   - Age
   - Roll Number
   - Marks

6.End

Program:

```
#include<iostream>
using namespace std; class student
{
public:
```

```
string n,add; int r,a;
void accept()
{

cout<<"Enter your name"<<endl; cin>>n;
cout<<"Enter your roll number"<<endl; cin>>r;
cout<<"Enter your age"<<endl; cin>>a;
cout<<"Enter your address"<<endl; cin>>add;
}
void display()
{
}
};
int main()
{
cout<<"Your name is:"<<n<<endl;
cout<<"Your roll number is:"<<r<<endl; cout<<"Your age is: "<<a<<endl;
cout<<"Your address is: "<<add<<endl;
student s; s.accept();
s.display(); return 0;
}
```

Output:

```
C:\Users\JSPM\Desktop\stud

Enter your name
Sachin
Enter your roll number
15
Enter your age
25
Enter your address
Pune
Your name is:Sachin
Your roll number is:15
Your age is: 25
Your address is: Pune

--------------------------------
Process exited after 10.79 seconds with return value 0
Press any key to continue . . .
```

Conclusion

This program serves as an excellent introductory exercise in C++ for practicing the fundamentals of Object-Oriented Programming (OOP), such as classes, objects, encapsulation, and input/output handling. It provides a clear structure and lays the foundation for more complex systems that manage larger datasets.

References:

- "The C++ Programming Language" by Bjarne Stroustrup
- "Object-Oriented Programming with C++" by E. Balagurusamy
- "C++ Primer" by Stanley B. Lippman, Josée Lajoie, and Barbara E. Moo

Lab Exercise-04

Title:

Write a C++ program to swap two numbers with the help of object-oriented concepts.

Aim:

The aim of this lab exercise is to implement a C++ program that swaps.

Theory:

Swapping is the process of exchanging values between two variables. If we have two numbers, a and b, swapping ensures that the value of a is stored in b and vice versa.

Example of Swapping:

Before Swapping:

a = 5, b = 10

After Swapping a = 10, b = 5

Swapping is commonly used in sorting algorithms, data manipulation, and temporary value storage in various computational tasks.

C++ Programming Constructs

- Classes & Objects: Used to encapsulate data and behavior related to swapping.
- Encapsulation: Data members (numbers) and methods (swap function) are encapsulated within a class.
- Functions: Used to implement swapping logic.

Methodology:

- Start by including necessary header file
- Create a class
- Implementing a member function swap() to swap values.
- Displaying numbers before and after swapping.
- Using an object of the class to call the functions and perform swapping

Algorithm:

1. Start.
2. Declare a class Swapper with two private data members (num1, num2).
3. Define a function swap() that:

- Uses a temporary variable to swap num1 and num2.

4. Define a function display() to print the numbers.
5. In the main() function:

- Accept user input for two numbers.
- Create an object of Swapper.
- Display numbers before swapping.
- Call swap() to swap the values.
- Display numbers after swapping.

6. End the program.

Program

```
#include <iostream>
```

```
using namespace std;
class swapper
{
public:
int a,b,t; void swap()
{
cout << "Enter two numbers: "; cin >>a>>b;
t = a;
a = b;
b = t;
}
void display()
{
cout<<"Value of a:"<<a<<endl; cout<<"Value of b:"<<b<<endl;
}
};
int main()
{
swapper sw;
sw.swap();
cout<<"After Swapping: "<<endl; sw.display();
return 0;
```

}

Output:

```
C:\Users\JSPM\Desktop\swap

Enter two numbers: 10
15
After Swapping:
Value of a:15
Value of b:10

--------------------------------
Process exited after 3.102 seconds with return value 0
Press any key to continue . . .
```

Conclusion:

In this lab exercise, I have successfully written a C++ program to swap two numbers using object- oriented programming. I have learned how to use classes and objects and write clean, efficient C++ code to encapsulate the logic of swapping two numbers.

References:

- "The C++ Programming Language" by Bjarne Stroustrup
- "Object-Oriented Programming with C++" by E. Balagurusamy
- "C++ Primer" by Stanley B. Lippman, Josée Lajoie, and Barbara E. Moo

Lab Exercise-05

Title:

Write a C++ program to find area of a circle, triangle and rectangle using function overloading.

Aim:

To understand function overloading by implementing a program to find the area of a circle, triangle, and rectangle.

Theory:

Function overloading is a feature in C++ where multiple functions can have the same name but different parameters. The compiler differentiates these functions based on the number and types of parameters. This allows multiple implementations of a function name for different purposes

Methodology:

1. Define three functions named area with different parameters:
    - One for calculating the area of a circle (radius as input).
    - One for calculating the area of a rectangle (length and breadth as input).
    - One for calculating the area of a triangle (base and height as input).
2. Call these functions based on the given inputs

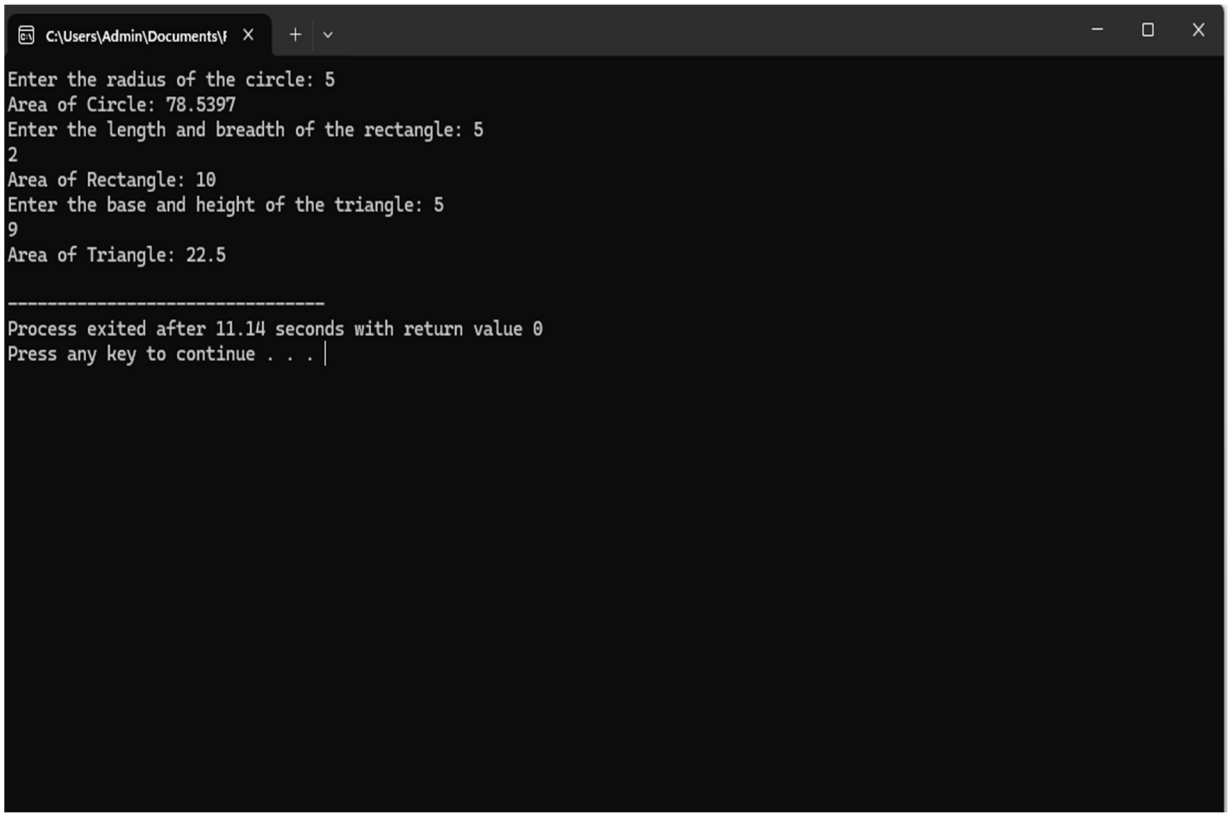
Algorithm:

1. Start.
2. Define a function area for a circle that takes the radius and returns the area.
3. Define a function area for a rectangle that takes length and breadth and returns the area.
4. Define a function area for a triangle that takes base and height and returns the area.
5. Call these functions with appropriate inputs.
6. Display the calculated area for each shape.
7. Stop.

Program:

```
#include <iostream>
using namespace std;
double area(double radius) {
   return 3.14159 * radius * radius;
}

double area(double length, double breadth) {
   return length * breadth;
}

double area(double base, double height, int) {
   return 0.5 * base * height;
}

int main() {
   double radius, length, breadth, base, height;

   cout << "Enter the radius of the circle: ";
   cin >> radius;
     cout << "Area of Circle: " << area(radius) << endl;
     cout << "Enter the length and breadth of the rectangle: ";
     cin >> length >> breadth;

       cout << "Area of Rectangle: " << area(length, breadth) << endl;

      cout << "Enter the base and height of the triangle: ";
      cin >> base >> height;
      cout << "Area of Triangle: " << area(base, height, 0) << endl;

return 0;
}
```

Output:



Conclusion:

Hence, we have studied function overloading in C++, where multiple functions with the same name but different parameters calculate the area of a circle, triangle, and rectangle, improving code reusability and readability.

References:

1. **"The C++ Programming Language"** by Bjarne Stroustrup
2. **"Object-Oriented Programming with C++"** by E. Balagurusamy
3. **"C++ Primer"** by Stanley B. Lippman, Josée Lajoie, and Barbara E. Moo

Lab Exercise-06

Title:

Write a C++ programs to design a BankAccount class with methods like deposit, withdraw, and getBalance. Ensure you have a constructor to set an initial balance.

Aim:

To implement a C++ program that defines a **BankAccount** class with attributes and methods for deposit, withdrawal, and balance checking, utilizing **object-oriented programming (OOP)** concepts like **encapsulation** and **constructors**.

Theory:

A **class** in C++ is a blueprint for objects that encapsulates data (attributes) and methods (functions). The **BankAccount** class represents a simple bank account system where:

- **Encapsulation** ensures that the account balance is private and only accessible via public methods.
- **A constructor** initializes the account balance when an object is created.
- **Member functions** allow depositing, withdrawing, and retrieving balance information.

Methodology:

- Define a BankAccount class with a private balance attribute.
- Implement a constructor to initialize the balance.
- Create methods:
- deposit(amount): Adds money to the balance.
- withdraw(amount): Deducts money if sufficient funds are available.
- getBalance(): Returns the current balance.
- Create a main() function to demonstrate deposit, withdrawal, and balance

- checking.

Algorithm:

1. Start.
2. Define a **BankAccount** class with:
3. A **private** attribute balance.
4. A **constructor** to initialize balance.
5. Methods: deposit(), withdraw(), and getBalance().
6. In the main() function:
7. Create a **BankAccount object** with an initial balance.
8. Perform **deposit** and **withdrawal** operations.
9. Display the final balance.
10. Stop.

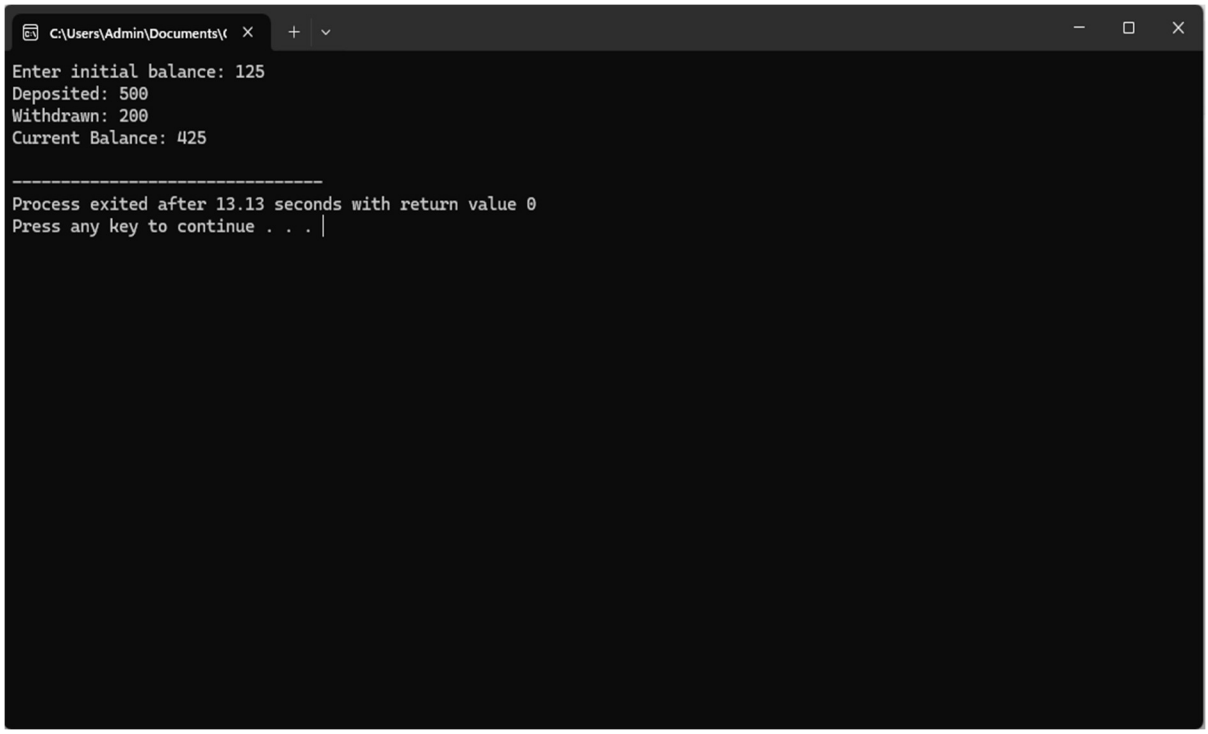
Program:

```
#include <iostream>
using namespace std;
class BankAccount
{
private:
   double balance;
public:
   BankAccount(double initialBalance) {
      if (initialBalance >= 0)
         balance = initialBalance;
      else {
         balance = 0;
```

```
        cout << "Invalid initial balance. Setting balance to 0." << endl;
    }
  }
  void deposit(double amount)
{
    if (amount > 0)
{
        balance += amount;
        cout << "Deposited: " << amount << endl;
    } else
 {
        cout << "Invalid deposit amount!" << endl;
    }
  }
  void withdraw(double amount) {
    if (amount > 0 && amount <= balance) {
        balance -= amount;
        cout << "Withdrawn: " << amount << endl;
    } else
{
        cout << "Invalid withdrawal amount or insufficient balance!" << endl;
    }
  }


  double getBalance() const {
    return balance;
  }
```

```
};

int main() {
    double initialAmount;
    cout << "Enter initial balance: ";
    cin >> initialAmount;
    BankAccount account(initialAmount);
    account.deposit(500);
    account.withdraw(200);
    cout << "Current Balance: " << account.getBalance() << endl;

    return 0;
}
```

Output:



Conclusion:

This program successfully demonstrates the **BankAccount** class implementation in C++ using **OOP concepts** such as **encapsulation**, **constructors**, and **member functions**. Key takeaways include:

1. **Encapsulation** protects sensitive data like the account balance by restricting direct access.
2. **Constructors** ensure that an initial balance is set when an account is created.
3. **Member functions** provide controlled access to deposit, withdraw, and check the balance.
4. The program models a **real-world banking system**, reinforcing **practical problem-solving skills** using OOP.

References:

- "The C++ Programming Language" by Bjarne Stroustrup
- "Object-Oriented Programming with C++" by E. Balagurusamy
- "C++ Primer" by Stanley B. Lippman, Josée Lajoie, and Barbara E. Moo

Lab Exercise-07

Title:

Write a C++ program to create a base class Account with attributes accountNumber and balance. Derive classes SavingsAccount and CurrentAccount with attributes interestRate and overdraftLimit. Add methods to deposit, withdraw, and display details.

Aim:

To implement a C++ program to create a base class Account with attributes and derived classes SavingsAccount and CurrentAccount using object-oriented programming concepts like inheritance and polymorphism.

Theory:

- **Inheritance:** Allows the creation of new classes based on existing ones.
- **Polymorphism:** Enables the use of methods in a derived class using base class pointers.
- **Encapsulation:** Protects data using private attributes and public methods.

Methodology:

1. Define a base class Account with account number and balance.
2. Derive classes SavingsAccount and CurrentAccount.
3. Add specific attributes like interest rate and overdraft limit.
4. Implement methods for depositing, withdrawing, and displaying account details.

Algorithm:

1. Start
2. Create a base class Account with account number and balance.
3. Implement methods for deposit, withdrawal, and displaying account details.
4. Derive SavingsAccount with an interest rate.

5. Derive CurrentAccount with an overdraft limit.
6. Perform necessary operations using objects of derived classes.
7. Display the results.
8. End

Program:

```
#include <iostream>

using namespace std;

class Account {

protected:

 int accountNumber;

 double balance;

public:

  Account(int accNum, double bal) : accountNumber(accNum), balance(bal) {}

  void deposit(double amount) {

   balance += amount;

    cout << "Deposited: " << amount << " | New Balance: " << balance << endl;

    }

   virtual void withdraw(double amount) {

   if (amount <= balance) {

   balance -= amount;

   cout << "Withdrawn: " << amount << " | Remaining Balance: " << balance << endl;

     } else {

    cout << "Insufficient funds!" << endl;

     }

    }

  virtual void displayDetails() {
```

```
 cout << "Account Number: " << accountNumber << " | Balance: " << balance << endl;
    }
};
class SavingsAccount : public Account { double interestRate;
public:
 SavingsAccount(int accNum, double bal, double rate) : Account(accNum, bal),
interestRate(rate) {}
 void displayDetails() override {
  cout << "[Savings Account] " ;
  Account::displayDetails();
 cout << "Interest Rate: " << interestRate << "%" << endl;
    }
};
class CurrentAccount : public Account {
double overdraftLimit;
public:
CurrentAccount(int accNum, double bal, double limit) : Account(accNum, bal),
overdraftLimit(limit) {}
 void withdraw(double amount) override {
 if (amount <= balance + overdraftLimit) {
 balance -= amount;
 cout << "Withdrawn: " << amount << " | Remaining Balance: " << balance << endl;
  }


Else
 {
```

```
 cout << "Exceeds overdraft limit!" << endl;
 }
   }
void displayDetails() override {
 cout << "[Current Account] ";
 Account::displayDetails();
 cout << "Overdraft Limit: " << overdraftLimit << endl;
 }
};
int main() {
 SavingsAccount savings(101, 5000.0, 3.5);
 CurrentAccount current(102, 3000.0, 1000.0);
  cout << "--- Account Details ---\n";
  savings.displayDetails();
  current.displayDetails();
 cout << "\n--- Transactions ---\n";
 savings.deposit(1000);
 savings.withdraw(2000);
 current.deposit(500);
 current.withdraw(3500);
 current.withdraw(500);
 return 0;
}
```

OUTPUT:

```
C:\Users\Admin\Documents\S

--- Account Details ---
[Savings Account] Account Number: 101 | Balance: 5000
Interest Rate: 3.5%
[Current Account] Account Number: 102 | Balance: 3000
Overdraft Limit: 1000

--- Transactions ---
Deposited: 1000 | New Balance: 6000
Withdrawn: 2000 | Remaining Balance: 4000
Deposited: 500 | New Balance: 3500
Withdrawn: 3500 | Remaining Balance: 0
Withdrawn: 500 | Remaining Balance: -500

--------------------------------
Process exited after 0.1779 seconds with return value 0
Press any key to continue . . .
```

Conclusion:

This program demonstrates the implementation of inheritance and polymorphism using a base class and derived classes in C++. It also showcases how to apply the concepts of encapsulation and data abstraction effectively.

References:

1. **"The C++ Programming Language"** by Bjarne Stroustrup
2. **"Object-Oriented Programming with C++"** by E. Balagurusamy
3. **"C++ Primer"** by Stanley B. Lippman, Josée Lajoie, and Barbara E. Moo

Lab Exercise-08

Title:

Write a C++ program to demonstrate pointers and virtual functions. Create a base class Shape with a virtual area() function and derive Circle and Rectangle classes. Use a base class pointer to call the area() method and display the result.

Aim:

To implement a C++ program that demonstrates pointers and virtual functions using a base class Shape with a virtual area() function, and derived classes Circle and Rectangle. Use a base class pointer to call the area() method and display the result.

Theory

- **Virtual Functions:** Enable runtime polymorphism by allowing derived class methods to override base class methods.
- **Polymorphism:** Allows a single function interface to represent different data types.
- **Pointers to Base Class:** Provide the flexibility to invoke derived class functions using base class pointers.

Methodology

1. Define a base class Shape with a virtual area() method.
2. Derive classes Circle and Rectangle.
3. Override the area() method in both derived classes.
4. Use a base class pointer to call the area() method.
5. Display the calculated areas.

Algorithm

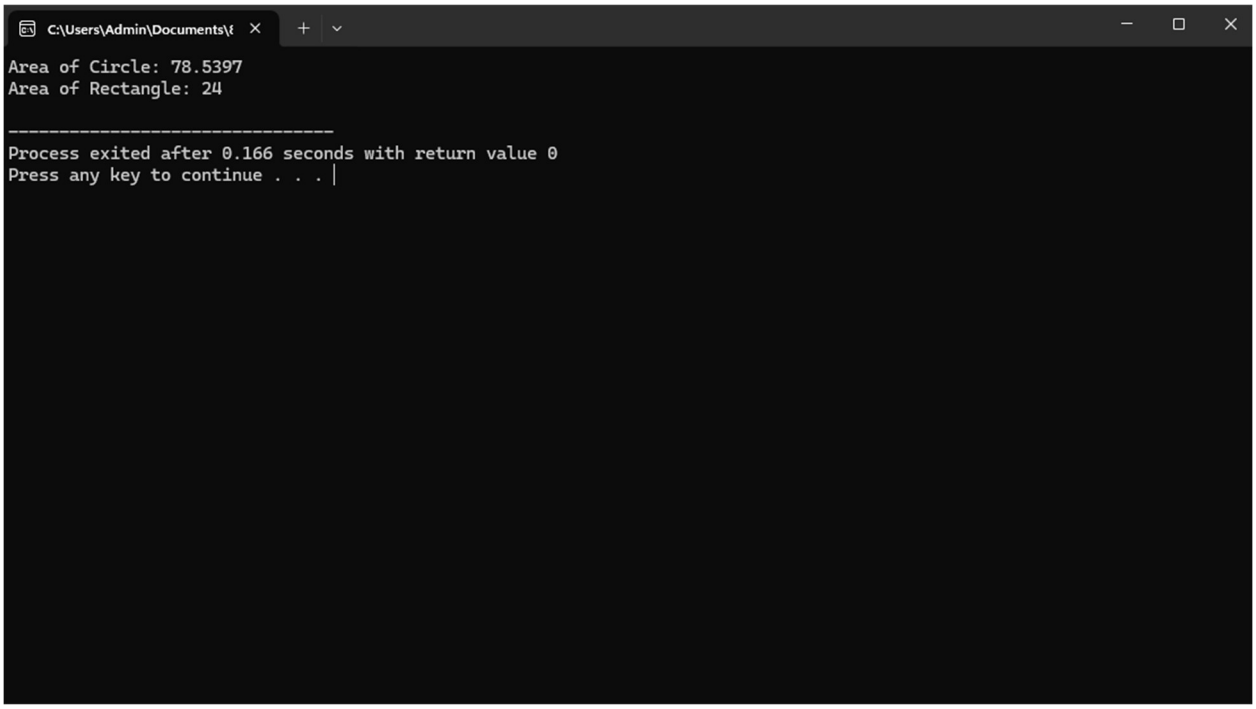
1. Start
2. Create a base class Shape with a virtual function area().
3. Derive Circle with radius as an attribute.
4. Derive Rectangle with length and width as attributes.
5. Override the area() function to calculate the respective areas.
6. Use a base class pointer to call the area() function.
7. Display the results.
8. End

Program:

```
#include <iostream>
using namespace std;
class Shape {
public:
virtual void area() {
cout << "Area of Shape (undefined)" << endl;
}
};
// Derived class Circle
class Circle : public Shape {
private:
double radius;
public:
Circle(double r) : radius(r) {}
void area() override {
cout << "Area of Circle: " << 3.14159 * radius * radius << endl;
```

```
}
};
class Rectangle : public Shape {
private:
double length, width;
public:
Rectangle(double l, double w) : length(l), width(w) {}
void area() override {
cout << "Area of Rectangle: " << length * width << endl;
}
};
int main() {
Shape* shapePtr; // Base class pointer
Circle circle(5.0);
Rectangle rectangle(4.0, 6.0);
shapePtr = &circle;
shapePtr->area();
shapePtr = &rectangle;
shapePtr->area();
return 0;
}
```

OUTPUT:



Conclusion

This program demonstrates the implementation of pointers and virtual functions using polymorphism. By using a base class pointer to call the area() method, the correct function is invoked at runtime, showcasing the power of virtual functions in C++.

References:

1. **"The C++ Programming Language"** by Bjarne Stroustrup
2. **"Object-Oriented Programming with C++"** by E. Balagurusamy
3. **"C++ Primer"** by Stanley B. Lippman, Josée Lajoie, and Barbara E. Moo

Lab Exercise-09

Title:

Implement a C++ program with a function divide() that handles division by zero using exception handling. If a division by zero occurs, throw an exception and display an error message.

Aim:

Implement a C++ program with a function divide() that handles division by zero using exception handling. If a division by zero occurs, throw an exception and display an error message

Theory:

- Exception handling is a mechanism in C++ that allows a program to handle runtime errors, such as division by zero, without crashing. It provides a way to detect, report, and handle runtime errors gracefully.
  - Division by zero is undefined in mathematics.
  - If a division by zero occurs in a program, it can lead to:
    - Program crash
    - Undefined behavior
    - Unexpected results

Methodology:

Function divide():

- Takes two arguments (dividend and divisor).
- Checks if the divisor is zero.
  - If zero, throws an exception.
  - If non-zero, returns the division result.

main() Function:

- Calls the divide() function within a try block.

- If an exception is thrown, the catch block handles the exception and displays an error message.

Algorithm:

Step 1: Start

Step 2: Define a function divide(a, b)

- **Input:** Two floating-point numbers a and b
- If b = 0, throw an exception (throw)
- Else, return the result of a / b

Step 3: In the main() function:

- Declare two floating-point variables num1 and num2
- Prompt the user to input num1 and num2

Step 4: Use a try block

- Call divide(num1, num2)
- If successful, display the result

Step 5: Use a catch block

- If an exception is thrown, display an error message

Step 6: End

Program:

```
#include <iostream>
using namespace std;

int main() {
```

```
    double numerator, denominator, divide;
    cout << "Enter numerator: ";
    cin >> numerator;
    cout << "Enter denominator: ";
    cin >> denominator;
    try
  {
        // throw an exception if denominator is 0
        if (denominator == 0)
            throw 0;
        // not executed if denominator is 0
        divide = numerator / denominator;
        cout << numerator << " / " << denominator << " = " << divide << endl;
    }
    catch (int num_exception) {
        cout << "Error: Cannot divide by " << num_exception << endl;
    }
    return 0;
}
```

OUTPUT:

Enter numerator: 30

Enter denominator: 0

ERROR!

Error: Cannot divide by 0

Conclusion:

Exception handling in C++ allows:

- Clean and structured handling of runtime errors.
- Avoids program crashes due to critical errors like division by zero.
- Improves program reliability and user experience.

References:

1. **"The C++ Programming Language"** by Bjarne Stroustrup
2. **"Object-Oriented Programming with C++"** by E. Balagurusamy
3. **"C++ Primer"** by Stanley B. Lippman, Josée Lajoie, and Barbara E. Moo

Lab Exercise-10

Title:

Implement a C++ program to create a template function findMax() that returns the maximum of two values of any data type (e.g., int, float, string).

Aim:

To Implement a C++ program with a function divide() that handles division by zero using exception handling. If a division by zero occurs, throw an exception and display an error message

OOP LAB EXERCISE 03

Theory:

In C++, **templates** are used to create generic functions or classes that can work with any data type without rewriting the code for each specific type.

What is a Template?

- A **template** is a blueprint or formula for creating generic functions or classes.
- It allows writing a single function or class definition that works with different data types.
- The compiler generates the appropriate function or class definition based on the type used during function calls.

Types of Templates in C++:

1. **Function Template** – Used to create generic functions.
2. **Class Template** – Used to create generic classes.

   Syntax of Function Template

   ```
   template <typename T>
   T functionName(T arg1, T arg2) {
   ```

```
    // Function body
```

Algorithm:

Algorithm for findMax() Template Function

1. Start
2. Define Template Function
   - Use the template <typename T> to define a generic data type T.
3. Define Function Signature
   - Create a function findMax() that takes two parameters of type T.
4. Compare Values
   - If the first value is greater than the second value, return the first value.
   - Else, return the second value.
5. Main Function
   - Declare variables of different data types (e.g., int, float, string).
   - Call findMax() function with different data types.
   - Print the returned value.
6. Stop

Program:

```
#include <iostream>
#include <string>
```

```
using namespace std;

// Template function to find the maximum of two values
template <typename T>
T findMax(T a, T b) {
    return (a > b) ? a : b;
}

int main() {
    // Test with integers
    int int1 = 10, int2 = 20;
    cout << "Max of " << int1 << " and " << int2 << " is " << findMax(int1,
int2) << endl;

    // Test with floats
    float float1 = 5.5, float2 = 2.3;
    cout << "Max of " << float1 << " and " << float2 << " is " <<
findMax(float1, float2) << endl;

    // Test with strings
    string str1 = "Apple", str2 = "Orange";
    cout << "Max of \"" << str1 << "\" and \"" << str2 << "\" is \"" <<
findMax(str1, str2) << "\"" << endl;
```

```
    return 0;
}
```

Output

Max of 10 and 20 is 20

Max of 5.5 and 2.3 is 5.5

Max of "Apple" and "Orange" is "Orange"

Conclusion:

The findMax() template function in C++ allows for a flexible and reusable way to determine the maximum of two values of any data type, including integers, floats, and strings. By using **templates**, the same function can handle different data types without needing to write separate code for each type. This demonstrates the power and versatility of **generic programming** in C++.

References:

1. **"The C++ Programming Language"** by Bjarne Stroustrup
2. **"Object-Oriented Programming with C++"** by E. Balagurusamy
3. **"C++ Primer"** by Stanley B. Lippman, Josée Lajoie, and Barbara E. Moo

Lab Exercise-11

Title:

Implement a C++ program to read and display student details (name, age, grade) from a file. If the file doesn't exist, display an error. Also, write new student details to a file.

Aim:

To implement a C++ program that reads student details from a file and displays them. If the file is not found, an error message is shown. Additionally, the program allows writing new student details to the file.

Theory:

File handling in C++ allows reading and writing data to files using input and output stream classes. The ifstream class is used for reading, and ofstream is used for writing data.

- If a file does not exist, attempting to read from it results in an error.
- Writing data to a file ensures persistent storage beyond program execution.
- Handling file errors is crucial for ensuring smooth program execution.

Methodology:

1. Reading from File:
    - Open the file using ifstream.
    - Read student details (name, age, grade) line by line.
    - Display the details.
    - If the file is missing, show an error message.
2. Writing to File:
    - Open the file using ofstream.
    - Accept new student details from the user.

- Write the details to the file.

Algorithm:

Step 1: Start
Step 2: Open the file in read mode
Step 3: If the file does not exist, display an error message and proceed to writing mode
Step 4: If the file exists, read and display student details
Step 5: Prompt the user to enter new student details
Step 6: Open the file in write mode and append new details Step 7: Close the file
Step 8: End

Program

```
#include <iostream>

#include <fstream>

using namespace std;

void readStudentDetails() {

   ifstream inputFile("students.txt");

   if (!inputFile) {

      cout << "Error: File not found!" << endl;

      return;

   }

   string name;

   int age;

   char grade;

   cout << "Student Details:" << endl;

   while (inputFile >> name >> age >> grade) {
```

```
        cout << "Name: " << name << ", Age: " << age << ", Grade: " << grade << endl;
    }
    inputFile.close();
}

void writeStudentDetails() {
    ofstream outputFile("students.txt", ios::app);
    if (!outputFile) {
        cout << "Error: Unable to open file for writing!" << endl;
        return;
    }
    string name;
    int age;
    char grade;
    cout << "Enter student name: ";
    cin >> name;
    cout << "Enter age: ";
    cin >> age;
    cout << "Enter grade: ";
    cin >> grade;
    outputFile << name << " " << age << " " << grade << endl;

    outputFile.close();
    cout << "Details saved successfully!" << endl;
}
```

```
int main() {
    readStudentDetails();
    writeStudentDetails();
    return 0;
}
```

Output:

```
C:\Users\admin\Documents\f

Error: File not found!
Enter student name: swapnil
Enter age: 21
Enter grade: A+
Details saved successfully!

--------------------------------
Process exited after 21.81 seconds with return value 0
Press any key to continue . . .
```

Conclusion:

we have successfully demonstrated file handling in C++ by reading student details from a file and handling errors when the file is missing. Additionally, we have implemented functionality to write new student details to the file, ensuring persistent storage and efficient data management.

References:

- "The C++ Programming Language" by Bjarne Stroustrup
- "Object-Oriented Programming with C++" by E. Balagurusamy

- "C++ Primer" by Stanley B. Lippman, Josée Lajoie, and Barbara E. Moo